North Bengal International University

Lab Report: 02

Exp Name : Bubble sort Algorithm

Subject : Analysis of algorithm Sessional

Subject Code : CSE 0612-2212

Submitted By:

Name : Md. Tanvir Rahman Pranto
Dept : CSE(26)
Roll : 0712340012101005
Semester : 3rd Year, 5th semester.

Submitted To:

Name : Afsana Tasnim
Designation : (HOD)Lecturer
Dept : CSE
Date :

Task No: 01

Task Name: Bubble sort

Introduction: Bubble sort is a simple, comparison-based, in-place sorting algorithm that repeatedly passes through a list, compares adjacent items and swaps them if they are in the wrong order. Over repeated passes the largest (or smallest) elements "bubble" to their final positions.

Algorithm:

- Start with the full array.
- Compare the first and second elements. If they are out of order (depending on ascending/descending), swap them.
- Move one position right and compare the next adjacent pair; swap if needed.
- Continue steps 2–3 until you reach the end of the array — after this first pass the largest element will be at the end.
- Repeat passes over the array, but each time ignore the last element(s) that are already sorted (reduce the effective length by 1).
- Optionally: if a full pass makes no swaps, the array is already sorted — stop early (optimization).

Pseudocode:

```
bubbleSort(A[0..n-1])
   read Array← from user
      for i from 0 to n-2 do
         if A[i] > A[i+1] then
            swap A[i] and A[i+1]
            swapped ← true
         end if
      end for
      n ← n - 1
   until not swapped
end
```

code:

```c
#include <stdio.h>

int main() {
    int n, i, j, temp;
    int swap;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }


    printf("\nBefore sorting: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");


    for (i = 0; i < n - 1; i++) {
        swap = 0;
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }
        if (swap == 0) {
            break;
        }
    }


    printf("After sorting:  ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

output:

```
OUTPUT    TERMINAL

Enter the number of elements : 5
Enter 5 elements: 8 6 4 3 1

 Before sorting: 8 6 4 3 1
 After sorting: 1 3 4 6 8
PS C:\Users\Lenovo\Documents\5th semester\Analysis of Algorithm\lab>
```

Time complexity: Worst/Average = $O(n^2)$, Best (already sorted with early stop) = $O(n)$.

Conclusion: Bubble Sort is a simple sorting algorithm that repeatedly compares and swaps adjacent elements until the list becomes sorted. Though it is easy to understand and implement, its time complexity is $O(n^2)$, which makes it inefficient for large datasets. However, it is useful for learning the basics of sorting techniques and works well for small arrays.